UTEC Posgrado

UTEC Posgrado

UTEC Posgrado

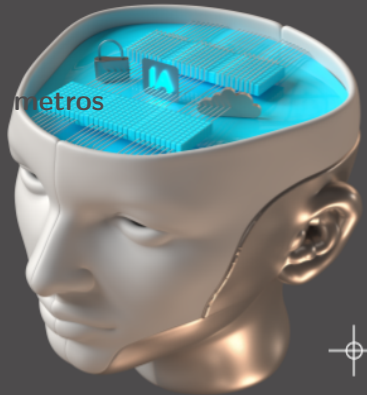
Una escuela para
Early Adopters

MACHINE LEARNING

BÚSQUEDA DE HIPERPARÁMETROS

Motivación

Contexto y necesidad **de la optimización de hiperparámetros**



Parámetros vs. hiperparámetros

Parámetros del modelo

- Se **aprenden** durante el entrenamiento

Hiperparámetros

- Se fijan **antes** del entrenamiento

Parámetros vs. hiperparámetros

Parámetros del modelo

- Se **aprenden** durante el entrenamiento
- Determinados por el algoritmo de optimización

Hiperparámetros

- Se fijan **antes** del entrenamiento
- No son aprendidos por el modelo

Parámetros vs. hiperparámetros

Parámetros del modelo

- Se **aprenden** durante el entrenamiento
- Determinados por el algoritmo de optimización
- Ejemplos: pesos w, sesgos b

Hiperparámetros

- Se fijan **antes** del entrenamiento
- No son aprendidos por el modelo
- Controlan la estructura y el proceso de aprendizaje

Parámetros vs. hiperparámetros

Parámetros del modelo

- Se **aprenden** durante el entrenamiento
- Determinados por el algoritmo de optimización
- Ejemplos: pesos w, sesgos b

Hiperparámetros

- Se fijan **antes** del entrenamiento
- No son aprendidos por el modelo
- Controlan la estructura y el proceso de aprendizaje

Modelo	Parámetros	Hiperparámetros
Regresión lineal	Pesos w_j	Regularización λ
SVM	Vectores soporte, α_i	C, γ (kernel RBF)
Red neuronal	Pesos, sesgos	Tasa de aprendizaje, capas
Árbol de decisión	Umbrales de corte	Profundidad máxima, min_samples

El problema de optimización de hiperparámetros

Objetivo: Encontrar la configuración de hiperparámetros $\boldsymbol{\lambda}^*$ que minimice el error de generalización.

El problema de optimización de hiperparámetros

Objetivo: Encontrar la configuración de hiperparámetros $\boldsymbol{\lambda}^*$ que minimice el error de generalización.

Formulación formal:

$$\boldsymbol{\lambda}^* = \arg\min_{\boldsymbol{\lambda} \in \Lambda} \mathcal{L}(\boldsymbol{\lambda})$$

El problema de optimización de hiperparámetros

Objetivo: Encontrar la configuración de hiperparámetros $\boldsymbol{\lambda}^*$ que minimice el error de generalización.

Formulación formal:

$$\boldsymbol{\lambda}^* = \arg\min_{\boldsymbol{\lambda} \in \Lambda} \mathcal{L}(\boldsymbol{\lambda})$$

donde $\mathcal{L}(\boldsymbol{\lambda})$ es el error de validación cruzada:

$$\mathcal{L}(\boldsymbol{\lambda}) = \text{CV-error}\big(\text{modelo}(\boldsymbol{\lambda}), \mathcal{D}\big)$$

El problema de optimización de hiperparámetros

Objetivo: Encontrar la configuración de hiperparámetros $\boldsymbol{\lambda}^*$ que minimice el error de generalización.

Formulación formal:

$$\boldsymbol{\lambda}^* = \arg\min_{\boldsymbol{\lambda} \in \Lambda} \mathcal{L}(\boldsymbol{\lambda})$$

donde $\mathcal{L}(\boldsymbol{\lambda})$ es el error de validación cruzada:

$$\mathcal{L}(\boldsymbol{\lambda}) = \text{CV-error}\big(\text{modelo}(\boldsymbol{\lambda}), \mathcal{D}\big)$$

Características clave:

- Es un problema de **optimización de caja negra** (black-box)
- No tenemos una expresión analítica de $\mathcal{L}(\boldsymbol{\lambda})$
- Cada evaluación requiere **entrenar y validar** el modelo completo

¿ Por qué es difícil?

1. **No convexa**
   La superficie de error tiene múltiples mínimos locales

¿ Por qué es difícil?

1. **No convexa**
   La superficie de error tiene múltiples mínimos locales
2. **Costosa de evaluar**
   Cada evaluación $\mathcal{L}(\boldsymbol{\lambda})$ requiere entrenamiento completo con validación cruzada

¿ Por qué es difícil?

1. **No convexa**
   La superficie de error tiene múltiples mínimos locales
2. **Costosa de evaluar**
   Cada evaluación $\mathcal{L}(\boldsymbol{\lambda})$ requiere entrenamiento completo con validación cruzada
3. **Sin gradiente**
   No hay derivada analítica $\nabla_{\boldsymbol{\lambda}}\mathcal{L}$

¿ Por qué es difícil?

1. **No convexa**
   La superficie de error tiene múltiples mínimos locales
2. **Costosa de evaluar**
   Cada evaluación $\mathcal{L}(\boldsymbol{\lambda})$ requiere entrenamiento completo con validación cruzada
3. **Sin gradiente**
   No hay derivada analítica $\nabla_{\boldsymbol{\lambda}}\mathcal{L}$
4. **Tipos mixtos**
   Hiperparámetros continuos (C), discretos (número de capas), categóricos (tipo de kernel)

¿ Por qué es difícil?

1. **No convexa**
   La superficie de error tiene múltiples mínimos locales
2. **Costosa de evaluar**
   Cada evaluación $\mathcal{L}(\boldsymbol{\lambda})$ requiere entrenamiento completo con validación cruzada
3. **Sin gradiente**
   No hay derivada analítica $\nabla_{\boldsymbol{\lambda}}\mathcal{L}$
4. **Tipos mixtos**
   Hiperparámetros continuos (C), discretos (número de capas), categóricos (tipo de kernel)
5. **Maldición de la dimensionalidad**
   Con d hiperparámetros y m valores cada uno: m^d combinaciones

¿ Por qué es difícil?

1. **No convexa**
   La superficie de error tiene múltiples mínimos locales
2. **Costosa de evaluar**
   Cada evaluación $\mathcal{L}(\boldsymbol{\lambda})$ requiere entrenamiento completo con validación cruzada
3. **Sin gradiente**
   No hay derivada analítica $\nabla_{\boldsymbol{\lambda}}\mathcal{L}$
4. **Tipos mixtos**
   Hiperparámetros continuos (C), discretos (número de capas), categóricos (tipo de kernel)
5. **Maldición de la dimensionalidad**
   Con d hiperparámetros y m valores cada uno: m^d combinaciones
6. **Ruidosa**
   La estimación de $\mathcal{L}$ varía según los folds de CV

Espacio de búsqueda Λ

El **espacio de búsqueda** Λ define el rango de valores posibles para cada hiperparámetro.

Espacio de búsqueda Λ

El **espacio de búsqueda** Λ define el rango de valores posibles para cada hiperparámetro.

Tipos de espacios:

- **Continuo acotado:** $\lambda_j \in [a, b]$ (ej. tasa de aprendizaje $\in [10^{-5}, 1]$)
- **Discreto:** $\lambda_j \in \{v_1, v_2, \ldots, v_k\}$ (ej. número de capas $\in \{1, 2, 3, 4\}$)
- **Categórico:** $\lambda_j \in \{\text{lineal}, \text{RBF}, \text{polinomial}\}$

Espacio de búsqueda Λ

El **espacio de búsqueda** Λ define el rango de valores posibles para cada hiperparámetro.

Tipos de espacios:

- **Continuo acotado:** $\lambda_j \in [a, b]$ (ej. tasa de aprendizaje $\in [10^{-5}, 1]$)
- **Discreto:** $\lambda_j \in \{v_1, v_2, \ldots, v_k\}$ (ej. número de capas $\in \{1, 2, 3, 4\}$)
- **Categórico:** $\lambda_j \in \{\text{lineal}, \text{RBF}, \text{polinomial}\}$

Ejemplo — SVM con kernel RBF (nuestro ejemplo a lo largo de la sesión):

Hiperparámetro	Tipo	Rango
C (regularización)	Continuo (log-escala)	[0,01, 100]
γ (ancho del kernel)	Continuo (log-escala)	[0,001, 10]

Validación cruzada K-Fold

Definición: Dividir los datos $\mathcal{D}$ en K particiones (folds) de tamaño similar.

Validación cruzada K-Fold

Definición: Dividir los datos $\mathcal{D}$ en K particiones (folds) de tamaño similar.

Procedimiento: Para cada fold $k = 1, \ldots, K$:

- Entrenar el modelo con los $K-1$ folds restantes (datos $\mathcal{D}_{-k}$)
- Evaluar en el fold k (datos $\mathcal{D}_k$)

Validación cruzada K-Fold

Definición: Dividir los datos $\mathcal{D}$ en K particiones (folds) de tamaño similar.

Procedimiento: Para cada fold $k = 1, \ldots, K$:

- Entrenar el modelo con los $K-1$ folds restantes (datos $\mathcal{D}_{-k}$)
- Evaluar en el fold k (datos $\mathcal{D}_k$)

Fórmula:

$$\mathrm{CV}(\boldsymbol{\lambda}) = \frac{1}{K} \sum_{k=1}^{K} \mathcal{L}\big(\mathrm{modelo}_{-k}(\boldsymbol{\lambda}),\ \mathcal{D}_k\big)$$

Validación cruzada K-Fold

Definición: Dividir los datos $\mathcal{D}$ en K particiones (folds) de tamaño similar.

Procedimiento: Para cada fold $k = 1, \ldots, K$:

- Entrenar el modelo con los $K-1$ folds restantes (datos $\mathcal{D}_{-k}$)
- Evaluar en el fold k (datos $\mathcal{D}_k$)

Fórmula:

$$\mathrm{CV}(\boldsymbol{\lambda}) = \frac{1}{K} \sum_{k=1}^{K} \mathcal{L}\big(\mathrm{modelo}_{-k}(\boldsymbol{\lambda}),\ \mathcal{D}_k\big)$$

$K = 5$ **folds**

Iter 1:	Val	Train	Train	Train	Train
Iter 2:	Train	Val	Train	Train	Train
Iter 3:	Train	Train	Val	Train	Train
Iter 4:	Train	Train	Train	Val	Train
Iter 5:	Train	Train	Train	Train	Val

Validación cruzada — Ejemplo con $K = 3$

Dataset: 12 observaciones, SVM-RBF con $C = 1$, $\gamma = 0{,}1$.

Iteración	Train (8 obs.)	Validación (4 obs.)	Accuracy
$k = 1$	Obs. $\{5, 6, 7, 8, 9, 10, 11, 12\}$	Obs. $\{1, 2, 3, 4\}$	0.83
$k = 2$	Obs. $\{1, 2, 3, 4, 9, 10, 11, 12\}$	Obs. $\{5, 6, 7, 8\}$	0.87
$k = 3$	Obs. $\{1, 2, 3, 4, 5, 6, 7, 8\}$	Obs. $\{9, 10, 11, 12\}$	0.85

Validación cruzada — Ejemplo con $K = 3$

Dataset: 12 observaciones, SVM-RBF con $C = 1$, $\gamma = 0{,}1$.

Iteración	Train (8 obs.)	Validación (4 obs.)	Accuracy
$k = 1$	Obs. $\{5, 6, 7, 8, 9, 10, 11, 12\}$	Obs. $\{1, 2, 3, 4\}$	0.83
$k = 2$	Obs. $\{1, 2, 3, 4, 9, 10, 11, 12\}$	Obs. $\{5, 6, 7, 8\}$	0.87
$k = 3$	Obs. $\{1, 2, 3, 4, 5, 6, 7, 8\}$	Obs. $\{9, 10, 11, 12\}$	0.85

Accuracy promedio (3-fold CV):

$$\text{CV}(C{=}1,\ \gamma{=}0{,}1) = \frac{0{,}83 + 0{,}87 + 0{,}85}{3} = 0{,}85$$

Validación cruzada — Ejemplo con $K = 3$

Dataset: 12 observaciones, SVM-RBF con $C = 1$, $\gamma = 0{,}1$.

Iteración	Train (8 obs.)	Validación (4 obs.)	Accuracy
$k = 1$	Obs. $\{5, 6, 7, 8, 9, 10, 11, 12\}$	Obs. $\{1, 2, 3, 4\}$	0.83
$k = 2$	Obs. $\{1, 2, 3, 4, 9, 10, 11, 12\}$	Obs. $\{5, 6, 7, 8\}$	0.87
$k = 3$	Obs. $\{1, 2, 3, 4, 5, 6, 7, 8\}$	Obs. $\{9, 10, 11, 12\}$	0.85

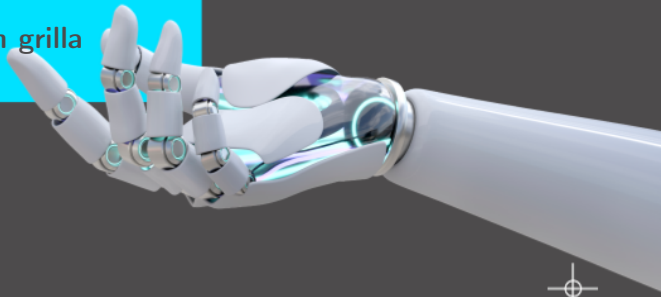
Accuracy promedio (3-fold CV):

$$\text{CV}(C{=}1,\ \gamma{=}0{,}1) = \frac{0{,}83 + 0{,}87 + 0{,}85}{3} = 0{,}85$$

Este valor de 0,85 es la **estimación del rendimiento** para esa configuración de hiperparámetros. Queremos encontrar la configuración que **maximice** este valor.

Desarrollo

Grid Search **Búsqueda exhaustiva en grilla**



Grid Search — Concepto

Idea: Evaluar **todas** las combinaciones del producto cartesiano de los valores candidatos para cada hiperparámetro.

- Definir una **grilla discreta** de valores para cada hiperparámetro

Grid Search — Concepto

Idea: Evaluar **todas** las combinaciones del producto cartesiano de los valores candidatos para cada hiperparámetro.

- Definir una **grilla discreta** de valores para cada hiperparámetro
- Evaluar el **producto cartesiano** $\Lambda_1 \times \Lambda_2 \times \cdots \times \Lambda_d$

Grid Search — Concepto

Idea: Evaluar **todas** las combinaciones del producto cartesiano de los valores candidatos para cada hiperparámetro.

- Definir una **grilla discreta** de valores para cada hiperparámetro
- Evaluar el **producto cartesiano** $\Lambda_1 \times \Lambda_2 \times \cdots \times \Lambda_d$
- Para cada combinación, calcular el **CV-score**

Grid Search — Concepto

Idea: Evaluar **todas** las combinaciones del producto cartesiano de los valores candidatos para cada hiperparámetro.

- Definir una **grilla discreta** de valores para cada hiperparámetro
- Evaluar el **producto cartesiano** $\Lambda_1 \times \Lambda_2 \times \cdots \times \Lambda_d$
- Para cada combinación, calcular el **CV-score**
- Seleccionar la combinación con **mejor rendimiento**

Grid Search — Concepto

Idea: Evaluar **todas** las combinaciones del producto cartesiano de los valores candidatos para cada hiperparámetro.

- Definir una **grilla discreta** de valores para cada hiperparámetro
- Evaluar el **producto cartesiano** $\Lambda_1 \times \Lambda_2 \times \cdots \times \Lambda_d$
- Para cada combinación, calcular el **CV-score**
- Seleccionar la combinación con **mejor rendimiento**

Propiedades:

- Simple de implementar y entender
- Fácilmente **paralelizable** (evaluaciones independientes)
- **Garantiza** encontrar el mejor punto *dentro de la grilla*

Grid Search — Algoritmo formal

Algoritmo 1: Grid Search

Entrada: Grilla $\Lambda_1 \times \Lambda_2 \times \cdots \times \Lambda_d$, modelo $\mathcal{M}$, datos $\mathcal{D}$, número de folds K
Salida : Mejor configuración $\boldsymbol{\lambda}^*$

```
mejor_score ← −∞
foreach λ = (λ_1, ..., λ_d) ∈ Λ_1 × Λ_2 × ··· × Λ_d do
    score ← CV(M, λ, D, K)
    if score > mejor_score then
        mejor_score ← score
        λ* ← λ
    end
end
return λ*, mejor_score
```

Complejidad: $|\Lambda_1| \times |\Lambda_2| \times \cdots \times |\Lambda_d| \times K$ entrenamientos.

Grid Search — Definición de la grilla

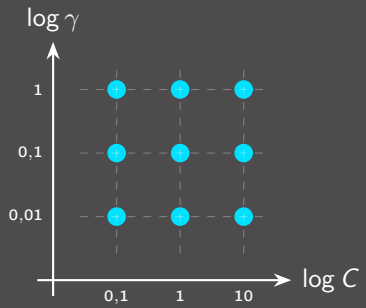
Ejemplo: SVM-RBF con dos hiperparámetros.

- $C \in \{0{,}1,\ 1,\ 10\}$
- $\gamma \in \{0{,}01,\ 0{,}1,\ 1\}$

Total de combinaciones:

$$3 \times 3 = 9 \text{ evaluaciones}$$

Cada una con 3-fold CV:
$9 \times 3 = 27$ entrenamientos.



Grid Search — Ejemplo paso a paso (1/2)

SVM-RBF, 3-fold CV. Primeras 3 combinaciones ($C = 0{,}1$):

Combinación 1: $C = 0{,}1$, $\gamma = 0{,}01$

Fold 1	Fold 2	Fold 3	**Media**
0.62	0.68	0.65	**0.65**

Grid Search — Ejemplo paso a paso (1/2)

SVM-RBF, 3-fold CV. Primeras 3 combinaciones ($C = 0{,}1$):

Combinación 1: $C = 0{,}1$, $\gamma = 0{,}01$

Fold 1	Fold 2	Fold 3	**Media**
0.62	0.68	0.65	**0.65**

Combinación 2: $C = 0{,}1$, $\gamma = 0{,}1$

Fold 1	Fold 2	Fold 3	**Media**
0.70	0.75	0.71	**0.72**

Grid Search — Ejemplo paso a paso (1/2)

SVM-RBF, 3-fold CV. Primeras 3 combinaciones ($C = 0{,}1$):

Combinación 1: $C = 0{,}1$, $\gamma = 0{,}01$

Fold 1	Fold 2	Fold 3	**Media**
0.62	0.68	0.65	**0.65**

Combinación 2: $C = 0{,}1$, $\gamma = 0{,}1$

Fold 1	Fold 2	Fold 3	**Media**
0.70	0.75	0.71	**0.72**

Combinación 3: $C = 0{,}1$, $\gamma = 1$

Fold 1	Fold 2	Fold 3	**Media**
0.55	0.60	0.59	**0.58**

$\gamma = 1$ es demasiado grande $\Rightarrow$ sobreajuste al ruido del kernel.

Grid Search — Ejemplo paso a paso (2/2)

Tabla completa de resultados (9 combinaciones, 3-fold CV):

C	γ	Fold 1	Fold 2	Fold 3	Media
0.1	0.01	0.62	0.68	0.65	0.65
0.1	0.1	0.70	0.75	0.71	0.72
0.1	1	0.55	0.60	0.59	0.58
1	0.01	0.76	0.80	0.78	0.78
1	0.1	0.83	0.87	0.85	**0.85**
1	1	0.68	0.72	0.70	0.70
10	0.01	0.73	0.77	0.75	0.75
10	0.1	0.80	0.84	0.82	0.82
10	1	0.61	0.65	0.63	0.63

- Mejor: $C^* = 1$, $\gamma^* = 0{,}1$, accuracy $= 0{,}85$

Grid Search — Mapa de calor de resultados

Mejor: $C=1,\ \gamma=0{,}1$

	$C = 0{,}1$	$C = 1$	$C = 10$
$\gamma = 1$	0.58	0.70	0.63
$\gamma = 0{,}1$	0.72	0.85	0.82
$\gamma = 0{,}01$	0.65	0.78	0.75

Grid Search — Ventajas y desventajas

Ventajas

- **Simple:** fácil de implementar y entender

Desventajas

Grid Search — Ventajas y desventajas

Ventajas

- **Simple:** fácil de implementar y entender
- **Paralelizable:** todas las evaluaciones son independientes

Desventajas

Grid Search — Ventajas y desventajas

Ventajas

- **Simple:** fácil de implementar y entender
- **Paralelizable:** todas las evaluaciones son independientes
- **Reproducible:** resultados determinísticos

Desventajas

Grid Search — Ventajas y desventajas

Ventajas

- **Simple:** fácil de implementar y entender
- **Paralelizable:** todas las evaluaciones son independientes
- **Reproducible:** resultados determinísticos
- **Garantía:** encuentra el mejor punto *dentro de la grilla*

Desventajas

Grid Search — Ventajas y desventajas

Ventajas

- **Simple:** fácil de implementar y entender
- **Paralelizable:** todas las evaluaciones son independientes
- **Reproducible:** resultados determinísticos
- **Garantía:** encuentra el mejor punto *dentro de la grilla*

Desventajas

- **Maldición de la dimensionalidad:** m^d combinaciones crecen exponencialmente

Grid Search — Ventajas y desventajas

Ventajas

- **Simple:** fácil de implementar y entender
- **Paralelizable:** todas las evaluaciones son independientes
- **Reproducible:** resultados determinísticos
- **Garantía:** encuentra el mejor punto *dentro de la grilla*

Desventajas

- **Maldición de la dimensionalidad:** m^d combinaciones crecen exponencialmente
- **Desperdicio de evaluaciones:** explora regiones irrelevantes

Grid Search — Ventajas y desventajas

Ventajas

- **Simple:** fácil de implementar y entender
- **Paralelizable:** todas las evaluaciones son independientes
- **Reproducible:** resultados determinísticos
- **Garantía:** encuentra el mejor punto *dentro de la grilla*

Desventajas

- **Maldición de la dimensionalidad:** m^d combinaciones crecen exponencialmente
- **Desperdicio de evaluaciones:** explora regiones irrelevantes
- **Resolución limitada:** el óptimo real puede estar *entre* los puntos de la grilla

Grid Search — Ventajas y desventajas

Ventajas

- **Simple:** fácil de implementar y entender
- **Paralelizable:** todas las evaluaciones son independientes
- **Reproducible:** resultados determinísticos
- **Garantía:** encuentra el mejor punto *dentro de la grilla*

Desventajas

- **Maldición de la dimensionalidad:** m^d combinaciones crecen exponencialmente
- **Desperdicio de evaluaciones:** explora regiones irrelevantes
- **Resolución limitada:** el óptimo real puede estar *entre* los puntos de la grilla
- **Ineficiente:** no aprende de evaluaciones previas

Grid Search — Ventajas y desventajas

Ventajas

- **Simple:** fácil de implementar y entender
- **Paralelizable:** todas las evaluaciones son independientes
- **Reproducible:** resultados determinísticos
- **Garantía:** encuentra el mejor punto *dentro de la grilla*

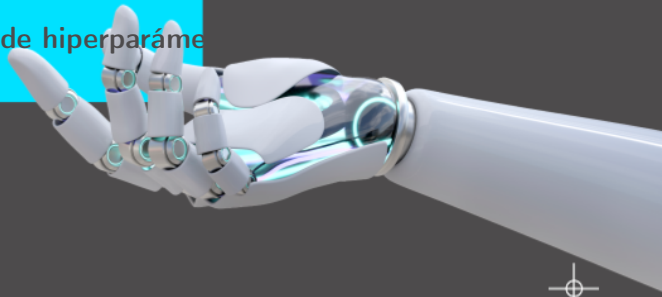
Desventajas

- **Maldición de la dimensionalidad:** m^d combinaciones crecen exponencialmente
- **Desperdicio de evaluaciones:** explora regiones irrelevantes
- **Resolución limitada:** el óptimo real puede estar *entre* los puntos de la grilla
- **Ineficiente:** no aprende de evaluaciones previas

Ejemplo: Con 5 hiperparámetros y 10 valores cada uno: $10^5 = 100{,}000$ evaluaciones.

Desarrollo

Random Search **Muestreo aleatorio de hiperparáme**



Random Search — Concepto

Idea: En lugar de evaluar *todas* las combinaciones, muestrear n combinaciones **aleatorias** de distribuciones definidas sobre cada hiperparámetro.

Random Search — Concepto

Idea: En lugar de evaluar *todas* las combinaciones, muestrear n combinaciones **aleatorias** de distribuciones definidas sobre cada hiperparámetro.

- **Basado en presupuesto:** el usuario fija el número n de evaluaciones

Random Search — Concepto

Idea: En lugar de evaluar *todas* las combinaciones, muestrear n combinaciones **aleatorias** de distribuciones definidas sobre cada hiperparámetro.

- **Basado en presupuesto:** el usuario fija el número n de evaluaciones
- Cada hiperparámetro se muestrea de una **distribución** (uniforme, log-uniforme, etc.)

Random Search — Concepto

Idea: En lugar de evaluar *todas* las combinaciones, muestrear n combinaciones **aleatorias** de distribuciones definidas sobre cada hiperparámetro.

- **Basado en presupuesto:** el usuario fija el número n de evaluaciones
- Cada hiperparámetro se muestrea de una **distribución** (uniforme, log-uniforme, etc.)
- Los puntos muestreados cubren el espacio de manera **más diversa** que una grilla

Random Search — Concepto

Idea: En lugar de evaluar *todas* las combinaciones, muestrear n combinaciones **aleatorias** de distribuciones definidas sobre cada hiperparámetro.

- **Basado en presupuesto:** el usuario fija el número n de evaluaciones
- Cada hiperparámetro se muestrea de una **distribución** (uniforme, log-uniforme, etc.)
- Los puntos muestreados cubren el espacio de manera **más diversa** que una grilla

Insight clave (Bergstra & Bengio, 2012):

Si solo algunos hiperparámetros son realmente importantes, Random Search es mucho más eficiente que Grid Search, porque explora **más valores únicos** por dimensión con el mismo presupuesto.

Random Search — Algoritmo formal

Algoritmo 2: Random Search

Entrada: Distribuciones $p(\lambda_1), \ldots, p(\lambda_d)$, presupuesto n, modelo $\mathcal{M}$, datos $\mathcal{D}$, folds K

Salida : Mejor configuración $\boldsymbol{\lambda}^*$

```
mejor_score ← −∞
for i = 1 to n do
    Muestrear λ_j^(i) ~ p(λ_j) para j = 1, ..., d
    λ^(i) ← (λ_1^(i), ..., λ_d^(i))
    score ← CV(M, λ^(i), D, K)
    if score > mejor_score then
        mejor_score ← score
        λ* ← λ^(i)
    end
end
return λ*, mejor_score
```

Random Search — Distribuciones de muestreo

¿ Cómo muestrear los hiperparámetros?

Random Search — Distribuciones de muestreo

¿ Cómo muestrear los hiperparámetros?

Uniforme: para rangos acotados simples

$$\lambda \sim \text{Uniform}(a, b)$$

Random Search — Distribuciones de muestreo

¿ Cómo muestrear los hiperparámetros?

Uniforme: para rangos acotados simples

$$\lambda \sim \text{Uniform}(a, b)$$

Log-Uniforme: para parámetros de escala (C, γ, learning rate)

$$\log(\lambda) \sim \text{Uniform}\big(\log(a), \log(b)\big) \quad \Longleftrightarrow \quad \lambda = e^u, \quad u \sim \text{Uniform}(\log a, \log b)$$

Random Search — Distribuciones de muestreo

¿ Cómo muestrear los hiperparámetros?

Uniforme: para rangos acotados simples

$$\lambda \sim \text{Uniform}(a, b)$$

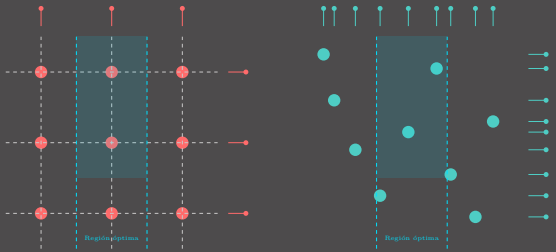
Log-Uniforme: para parámetros de escala (C, γ, learning rate)

$$\log(\lambda) \sim \text{Uniform}\big(\log(a), \log(b)\big) \quad \Longleftrightarrow \quad \lambda = e^u, \;\; u \sim \text{Uniform}(\log a, \log b)$$

¿ Por qué log-uniforme?

- Los hiperparámetros frecuentemente abarcan **órdenes de magnitud**
- $C \in [0{,}01,\ 100]$ abarca 4 órdenes: 10^{-2} a 10^{2}
- Una uniforme en $[0{,}01, 100]$ muestrearía casi todo en $[50, 100]$
- Log-uniforme asigna **igual probabilidad** a cada orden de magnitud

Grid vs. Random — Visualización



Observación clave: Con el **mismo presupuesto** (9 evaluaciones), Grid Search solo prueba 3 valores únicos por dimensión, mientras que Random Search prueba 9 valores únicos por dimensión.

Si un hiperparámetro no es importante, Grid Search **desperdicia** evaluaciones repitiendo valores en esa dimensión (Bergstra & Bengio, 2012).

Random Search — Ejemplo paso a paso

Mismo problema: SVM-RBF, $C \sim \text{LogU}(0{,}01, 100)$, $\gamma \sim \text{LogU}(0{,}001, 10)$, presupuesto $n = 9$.

Iter.	C	γ	CV Accuracy
1	0.35	0.520	0.68
2	5.21	0.080	0.83
3	0.08	0.003	0.60
4	45.70	1.230	0.55
5	1.42	0.150	**0.86**
6	0.72	0.040	0.74
7	12.30	0.210	0.80
8	2.85	0.007	0.71
9	0.19	0.950	0.57

Mejor: Iteración 5, $C^* = 1{,}42$, $\gamma^* = 0{,}15$, accuracy $= 0{,}86$

Random Search — Observación

Nota: Random Search obtuvo **mejor resultado** (0,86) que Grid Search (0,85) con el **mismo número de evaluaciones** ($n = 9$), porque exploró valores más finos en el espacio continuo de hiperparámetros.

Grid Search

- Solo evaluó $C \in \{0{,}1, 1, 10\}$
- Solo evaluó $\gamma \in \{0{,}01, 0{,}1, 1\}$
- Mejor: $C = 1$, $\gamma = 0{,}1 \rightarrow 0{,}85$

Random Search

- Exploró $C \in [0{,}01, 100]$ continuo
- Exploró $\gamma \in [0{,}001, 10]$ continuo
- Mejor: $C = 1{,}42$, $\gamma = 0{,}15 \rightarrow 0{,}86$

- La resolución más fina permitió encontrar una configuración intermedia que Grid Search no podía alcanzar.

Random Search — Análisis de probabilidad

¿ Qué probabilidad tenemos de encontrar una buena configuración?

Random Search — Análisis de probabilidad

¿ Qué probabilidad tenemos de encontrar una buena configuración?

Con n muestras independientes, la probabilidad de que al menos una caiga en el **top-p** del espacio es:

$$P(\text{encontrar top-}p) = 1 - (1 - p)^n$$

Random Search — Análisis de probabilidad

¿ Qué probabilidad tenemos de encontrar una buena configuración?

Con n muestras independientes, la probabilidad de que al menos una caiga en el **top-p** del espacio es:

$$P(\text{encontrar top-}p) = 1 - (1 - p)^n$$

	Fracción top-p		
***n* (evaluaciones)**	$p = 5\,\%$	$p = 10\,\%$	$p = 25\,\%$
9	0.37	0.61	0.92
20	0.64	0.88	0.997
40	0.87	0.985	≈ 1
60	0.95	0.998	≈ 1

Random Search — Análisis de probabilidad

¿ Qué probabilidad tenemos de encontrar una buena configuración?

Con n muestras independientes, la probabilidad de que al menos una caiga en el **top-p** del espacio es:

$$P(\text{encontrar top-}p) = 1 - (1 - p)^n$$

	Fracción top-p		
***n* (evaluaciones)**	$p = 5\,\%$	$p = 10\,\%$	$p = 25\,\%$
9	0.37	0.61	0.92
20	0.64	0.88	0.997
40	0.87	0.985	≈ 1
60	0.95	0.998	≈ 1

Regla práctica: Con **60 evaluaciones** aleatorias, tenemos un 95 % de probabilidad de encontrar una configuración en el mejor 5 % del espacio.

Random Search — Ventajas y desventajas

Ventajas

- **Eficiente:** mejor cobertura del espacio con presupuesto fijo

Desventajas

Random Search — Ventajas y desventajas

Ventajas

- **Eficiente:** mejor cobertura del espacio con presupuesto fijo
- **Fácil de paralelizar:** evaluaciones independientes

Desventajas

Random Search — Ventajas y desventajas

Ventajas

- **Eficiente:** mejor cobertura del espacio con presupuesto fijo
- **Fácil de paralelizar:** evaluaciones independientes
- **Mejor cobertura:** más valores únicos por dimensión

Desventajas

Random Search — Ventajas y desventajas

Ventajas

- **Eficiente:** mejor cobertura del espacio con presupuesto fijo
- **Fácil de paralelizar:** evaluaciones independientes
- **Mejor cobertura:** más valores únicos por dimensión
- **Sin maldición dimensional:** la complejidad no crece con d

Desventajas

Random Search — Ventajas y desventajas

Ventajas

- **Eficiente:** mejor cobertura del espacio con presupuesto fijo
- **Fácil de paralelizar:** evaluaciones independientes
- **Mejor cobertura:** más valores únicos por dimensión
- **Sin maldición dimensional:** la complejidad no crece con d
- **Análisis teórico:** garantías probabilísticas claras

Desventajas

Random Search — Ventajas y desventajas

Ventajas

- **Eficiente:** mejor cobertura del espacio con presupuesto fijo
- **Fácil de paralelizar:** evaluaciones independientes
- **Mejor cobertura:** más valores únicos por dimensión
- **Sin maldición dimensional:** la complejidad no crece con d
- **Análisis teórico:** garantías probabilísticas claras

Desventajas

- **No determinista:** resultados varían entre ejecuciones

Random Search — Ventajas y desventajas

Ventajas

- **Eficiente:** mejor cobertura del espacio con presupuesto fijo
- **Fácil de paralelizar:** evaluaciones independientes
- **Mejor cobertura:** más valores únicos por dimensión
- **Sin maldición dimensional:** la complejidad no crece con d
- **Análisis teórico:** garantías probabilísticas claras

Desventajas

- **No determinista:** resultados varían entre ejecuciones
- **Sin garantía absoluta:** puede no encontrar el óptimo

Random Search — Ventajas y desventajas

Ventajas

- **Eficiente:** mejor cobertura del espacio con presupuesto fijo
- **Fácil de paralelizar:** evaluaciones independientes
- **Mejor cobertura:** más valores únicos por dimensión
- **Sin maldición dimensional:** la complejidad no crece con d
- **Análisis teórico:** garantías probabilísticas claras

Desventajas

- **No determinista:** resultados varían entre ejecuciones
- **Sin garantía absoluta:** puede no encontrar el óptimo
- **No aprende:** no usa información de evaluaciones previas

Random Search — Ventajas y desventajas

Ventajas

- **Eficiente:** mejor cobertura del espacio con presupuesto fijo
- **Fácil de paralelizar:** evaluaciones independientes
- **Mejor cobertura:** más valores únicos por dimensión
- **Sin maldición dimensional:** la complejidad no crece con d
- **Análisis teórico:** garantías probabilísticas claras

Desventajas

- **No determinista:** resultados varían entre ejecuciones
- **Sin garantía absoluta:** puede no encontrar el óptimo
- **No aprende:** no usa información de evaluaciones previas
- **Depende de las distribuciones:** una mala elección de $p(\lambda)$ puede ser ineficiente

Random Search — Ventajas y desventajas

Ventajas

- **Eficiente:** mejor cobertura del espacio con presupuesto fijo
- **Fácil de paralelizar:** evaluaciones independientes
- **Mejor cobertura:** más valores únicos por dimensión
- **Sin maldición dimensional:** la complejidad no crece con d
- **Análisis teórico:** garantías probabilísticas claras

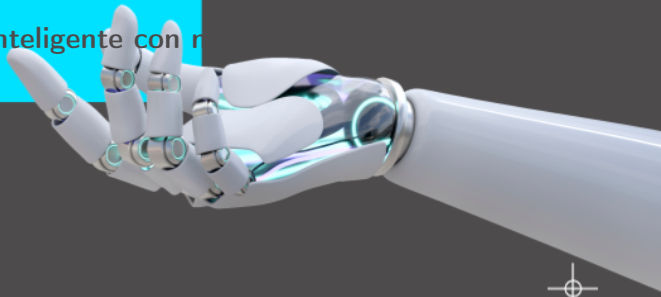
Desventajas

- **No determinista:** resultados varían entre ejecuciones
- **Sin garantía absoluta:** puede no encontrar el óptimo
- **No aprende:** no usa información de evaluaciones previas
- **Depende de las distribuciones:** una mala elección de $p(\lambda)$ puede ser ineficiente

Pregunta natural: ¿ Podemos usar la información de evaluaciones previas para **guiar** la búsqueda?

Desarrollo

Optimización Bayesiana **Búsqueda inteligente con r**



Optimización Bayesiana — Motivación

Grid Search y Random Search son métodos **no informados**: no aprenden de evaluaciones anteriores.

Optimización Bayesiana — Motivación

Grid Search y Random Search son métodos **no informados**: no aprenden de evaluaciones anteriores.

- Evaluación i: $(C = 0{,}35, \gamma = 0{,}52) \rightarrow$ accuracy $= 0{,}68$

Optimización Bayesiana — Motivación

Grid Search y Random Search son métodos **no informados**: no aprenden de evaluaciones anteriores.

- Evaluación i: $(C = 0{,}35, \gamma = 0{,}52) \rightarrow$ accuracy $= 0{,}68$
- Evaluación $i{+}1$: ¿ Dónde evaluar ahora?

Optimización Bayesiana — Motivación

Grid Search y Random Search son métodos **no informados**: no aprenden de evaluaciones anteriores.

- Evaluación i: $(C = 0{,}35, \gamma = 0{,}52) \rightarrow$ accuracy $= 0{,}68$
- Evaluación $i{+}1$: ¿ Dónde evaluar ahora?

Idea clave:

1. Construir un **modelo sustituto** (surrogate) de la función objetivo $f(\boldsymbol{\lambda})$
2. Usar este modelo para decidir **dónde evaluar a continuación**
3. Actualizar el modelo con la nueva observación
4. Repetir

Optimización Bayesiana — Motivación

Grid Search y Random Search son métodos **no informados**: no aprenden de evaluaciones anteriores.

- Evaluación i: $(C = 0{,}35, \gamma = 0{,}52) \rightarrow$ accuracy $= 0{,}68$
- Evaluación $i{+}1$: ¿ Dónde evaluar ahora?

Idea clave:

1. Construir un **modelo sustituto** (surrogate) de la función objetivo $f(\boldsymbol{\lambda})$
2. Usar este modelo para decidir **dónde evaluar a continuación**
3. Actualizar el modelo con la nueva observación
4. Repetir

▶ Referencia: Snoek, Larochelle & Adams (2012), "Practical Bayesian Optimization of Machine Learning Algorithms".

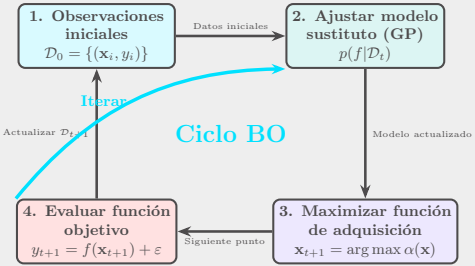
Optimización Bayesiana — Dos componentes

1. Modelo sustituto

- Aproxima la función objetivo $f(\boldsymbol{\lambda})$
- Comúnmente: **Proceso Gaussiano** (GP)
- Proporciona **media** (predicción) y **varianza** (incertidumbre)

2. Función de adquisición

- Decide **dónde** evaluar a continuación
- Balancea **exploración** (alta incertidumbre) vs. **explotación** (alto valor predicho)
- Fácil de optimizar (tiene fórmula



Proceso Gaussiano — Concepto

Un **Proceso Gaussiano** (GP) es una **distribución sobre funciones**:

$$f(\mathrm{x}) \sim \mathcal{GP}\big(m(\mathrm{x}),\ k(\mathrm{x}, \mathrm{x}')\big)$$

Proceso Gaussiano — Concepto

Un **Proceso Gaussiano** (GP) es una **distribución sobre funciones**:

$$f(\mathrm{x}) \sim \mathcal{GP}\big(m(\mathrm{x}),\ k(\mathrm{x}, \mathrm{x}')\big)$$

- $m(\mathrm{x})$: **función media** (prior sobre el valor esperado; comúnmente $m(\mathrm{x}) = 0$)

Proceso Gaussiano — Concepto

Un **Proceso Gaussiano** (GP) es una **distribución sobre funciones**:

$$f(\mathrm{x}) \sim \mathcal{GP}\big(m(\mathrm{x}),\ k(\mathrm{x}, \mathrm{x}')\big)$$

- $m(\mathrm{x})$: **función media** (prior sobre el valor esperado; comúnmente $m(\mathrm{x}) = 0$)
- $k(\mathrm{x}, \mathrm{x}')$: **función de covarianza** (kernel); mide similitud entre puntos

Proceso Gaussiano — Concepto

Un **Proceso Gaussiano** (GP) es una **distribución sobre funciones**:

$$f(\mathrm{x}) \sim \mathcal{GP}\big(m(\mathrm{x}),\ k(\mathrm{x},\mathrm{x}')\big)$$

- $m(\mathrm{x})$: **función media** (prior sobre el valor esperado; comúnmente $m(\mathrm{x}) = 0$)
- $k(\mathrm{x},\mathrm{x}')$: **función de covarianza** (kernel); mide similitud entre puntos

Propiedad fundamental:

Cualquier colección finita de valores de la función es **conjuntamente gaussiana**:

$$\begin{pmatrix} f(\mathrm{x}_1) \\ f(\mathrm{x}_2) \\ \vdots \\ f(\mathrm{x}_n) \end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix} m(\mathrm{x}_1) \\ m(\mathrm{x}_2) \\ \vdots \\ m(\mathrm{x}_n) \end{pmatrix}, \begin{pmatrix} k(\mathrm{x}_1,\mathrm{x}_1) & \cdots & k(\mathrm{x}_1,\mathrm{x}_n) \\ \vdots & \ddots & \vdots \\ k(\mathrm{x}_n,\mathrm{x}_1) & \cdots & k(\mathrm{x}_n,\mathrm{x}_n) \end{pmatrix} \right)$$

Proceso Gaussiano — Kernel RBF

El kernel más común es el **RBF** (Radial Basis Function):

$$k(\mathrm{x}, \mathrm{x}') = \sigma_f^2 \exp\left(-\frac{\|\mathrm{x} - \mathrm{x}'\|^2}{2\ell^2}\right)$$

Parámetros del kernel:

- σ_f^2: **varianza de señal** (amplitud)
- ℓ: **longitud de escala** (suavidad)
- ℓ grande $\rightarrow$ funciones suaves
- ℓ pequeño $\rightarrow$ funciones rápidas

Intuición:

Distancia	Covarianza
$\|\mathrm{x} - \mathrm{x}'\| = 0$	σ_f^2 (máxima)
$\|\mathrm{x} - \mathrm{x}'\| \ll \ell$	$\approx \sigma_f^2$ (alta)
$\|\mathrm{x} - \mathrm{x}'\| = \ell$	$\sigma_f^2 e^{-1/2} \approx 0{,}6\sigma_f^2$
$\|\mathrm{x} - \mathrm{x}'\| \gg \ell$	≈ 0 (baja)

Proceso Gaussiano — Predicción posterior

Dados n observaciones $\mathcal{D} = \{(\mathrm{x}_i, y_i)\}_{i=1}^{n}$, predecir en un nuevo punto x_*:

Proceso Gaussiano — Predicción posterior

Dados n observaciones $\mathcal{D} = \{(\mathrm{x}_i, y_i)\}_{i=1}^{n}$, predecir en un nuevo punto x_*:

Media posterior (predicción):

$$\mu(\mathrm{x}_*) = \mathrm{k}_*^\top \left[\mathrm{K} + \sigma_n^2 \mathrm{I}\right]^{-1} \mathrm{y}$$

Proceso Gaussiano — Predicción posterior

Dados n observaciones $\mathcal{D} = \{(\mathrm{x}_i, y_i)\}_{i=1}^n$, predecir en un nuevo punto x_*:

Media posterior (predicción):

$$\mu(\mathrm{x}_*) = \mathrm{k}_*^\top \left[\mathrm{K} + \sigma_n^2 \mathrm{I}\right]^{-1} \mathrm{y}$$

Varianza posterior (incertidumbre):

$$\sigma^2(\mathrm{x}_*) = k(\mathrm{x}_*, \mathrm{x}_*) - \mathrm{k}_*^\top \left[\mathrm{K} + \sigma_n^2 \mathrm{I}\right]^{-1} \mathrm{k}_*$$

Proceso Gaussiano — Predicción posterior

Dados n observaciones $\mathcal{D} = \{(\mathrm{x}_i, y_i)\}_{i=1}^n$, predecir en un nuevo punto x_*:

Media posterior (predicción):

$$\mu(\mathrm{x}_*) = \mathrm{k}_*^\top \left[\mathrm{K} + \sigma_n^2 \mathrm{I}\right]^{-1} \mathrm{y}$$

Varianza posterior (incertidumbre):

$$\sigma^2(\mathrm{x}_*) = k(\mathrm{x}_*, \mathrm{x}_*) - \mathrm{k}_*^\top \left[\mathrm{K} + \sigma_n^2 \mathrm{I}\right]^{-1} \mathrm{k}_*$$

donde:

- K: matriz de covarianza $n \times n$, $K_{ij} = k(\mathrm{x}_i, \mathrm{x}_j)$
- k_*: vector de covarianzas entre x_* y los datos, $[\mathrm{k}_*]_i = k(\mathrm{x}_*, \mathrm{x}_i)$
- σ_n^2: varianza del ruido de observación

Proceso Gaussiano — Predicción posterior

Dados n observaciones $\mathcal{D} = \{(\mathrm{x}_i, y_i)\}_{i=1}^n$, predecir en un nuevo punto x_*:

Media posterior (predicción):

$$\mu(\mathrm{x}_*) = \mathrm{k}_*^\top \left[\mathrm{K} + \sigma_n^2 \mathrm{I}\right]^{-1} \mathrm{y}$$

Varianza posterior (incertidumbre):

$$\sigma^2(\mathrm{x}_*) = k(\mathrm{x}_*, \mathrm{x}_*) - \mathrm{k}_*^\top \left[\mathrm{K} + \sigma_n^2 \mathrm{I}\right]^{-1} \mathrm{k}_*$$

donde:

- K: matriz de covarianza $n \times n$, $K_{ij} = k(\mathrm{x}_i, \mathrm{x}_j)$
- k_*: vector de covarianzas entre x_* y los datos, $[\mathrm{k}_*]_i = k(\mathrm{x}_*, \mathrm{x}_i)$
- σ_n^2: varianza del ruido de observación

Funciones de adquisición

La función de adquisición $\alpha(x)$ decide dónde evaluar a continuación: $x_{next} = \arg\max_{x \in \Lambda} \alpha(x)$

Tres funciones principales:

Funciones de adquisición

La función de adquisición $\alpha(\mathrm{x})$ decide dónde evaluar a continuación: $\mathrm{x}_{\mathrm{next}} = \arg\max_{\mathrm{x}\in\Lambda} \alpha(\mathrm{x})$

Tres funciones principales:

1. **Probability of Improvement (PI):**

$$\mathrm{PI}(\mathrm{x}) = P\big(f(\mathrm{x}) > f(\mathrm{x}^+)\big) = \Phi(Z)$$

Funciones de adquisición

La función de adquisición $\alpha(\mathrm{x})$ decide dónde evaluar a continuación: $\mathrm{x}_{\mathrm{next}} = \arg\max_{\mathrm{x}\in\Lambda} \alpha(\mathrm{x})$

Tres funciones principales:

1. **Probability of Improvement (PI):**

$$\mathrm{PI}(\mathrm{x}) = P\big(f(\mathrm{x}) > f(\mathrm{x}^+)\big) = \Phi(Z)$$

2. **Expected Improvement (EI):**

$$\mathrm{EI}(\mathrm{x}) = \mathbb{E}\big[\,\mathrm{máx}\,\big(f(\mathrm{x}) - f(\mathrm{x}^+),\ 0\big)\big]$$

Funciones de adquisición

La función de adquisición $\alpha(\mathrm{x})$ decide dónde evaluar a continuación: $\mathrm{x}_{\mathrm{next}} = \arg\max_{\mathrm{x}\in\Lambda} \alpha(\mathrm{x})$

Tres funciones principales:

1. **Probability of Improvement (PI):**

$$\mathrm{PI}(\mathrm{x}) = P\big(f(\mathrm{x}) > f(\mathrm{x}^+)\big) = \Phi(Z)$$

2. **Expected Improvement (EI):**

$$\mathrm{EI}(\mathrm{x}) = \mathbb{E}\big[\,\mathrm{máx}\,\big(f(\mathrm{x}) - f(\mathrm{x}^+),\ 0\big)\big]$$

3. **Upper Confidence Bound (UCB):**

$$\mathrm{UCB}(\mathrm{x}) = \mu(\mathrm{x}) + \kappa \cdot \sigma(\mathrm{x})$$

Funciones de adquisición

La función de adquisición $\alpha(\mathrm{x})$ decide dónde evaluar a continuación: $\mathrm{x}_{\mathrm{next}} = \arg\max_{\mathrm{x}\in\Lambda} \alpha(\mathrm{x})$

Tres funciones principales:

1. **Probability of Improvement (PI):**

$$\mathrm{PI}(\mathrm{x}) = P\big(f(\mathrm{x}) > f(\mathrm{x}^+)\big) = \Phi(Z)$$

2. **Expected Improvement (EI):**

$$\mathrm{EI}(\mathrm{x}) = \mathbb{E}\big[\max\big(f(\mathrm{x}) - f(\mathrm{x}^+),\ 0\big)\big]$$

3. **Upper Confidence Bound (UCB):**

$$\mathrm{UCB}(\mathrm{x}) = \mu(\mathrm{x}) + \kappa \cdot \sigma(\mathrm{x})$$

donde $f(\mathrm{x}^+)$ es el mejor valor observado hasta ahora, y $Z = \frac{\mu(\mathrm{x}) - f(\mathrm{x}^+)}{\sigma(\mathrm{x})}$.

Expected Improvement (EI) — Fórmula cerrada

La función de adquisición más popular. Fórmula cerrada:

$$\mathrm{EI}(\mathrm{x}) = \begin{cases} \left(\mu(\mathrm{x}) - f(\mathrm{x}^+)\right)\Phi(Z) + \sigma(\mathrm{x})\,\phi(Z) & \text{si } \sigma(\mathrm{x}) > 0 \\ 0 & \text{si } \sigma(\mathrm{x}) = 0 \end{cases}$$

Expected Improvement (EI) — Fórmula cerrada

La función de adquisición más popular. Fórmula cerrada:

$$\mathrm{EI}(\mathrm{x}) = \begin{cases} \left(\mu(\mathrm{x}) - f(\mathrm{x}^+)\right)\Phi(Z) + \sigma(\mathrm{x})\,\phi(Z) & \text{si } \sigma(\mathrm{x}) > 0 \\ 0 & \text{si } \sigma(\mathrm{x}) = 0 \end{cases}$$

donde:

$$Z = \frac{\mu(\mathrm{x}) - f(\mathrm{x}^+)}{\sigma(\mathrm{x})}$$

- $\phi(\cdot)$: PDF de la normal estándar
- $\Phi(\cdot)$: CDF de la normal estándar
- $f(\mathrm{x}^+)$: mejor valor observado hasta ahora

Expected Improvement (EI) — Fórmula cerrada

La función de adquisición más popular. Fórmula cerrada:

$$\mathrm{EI}(\mathrm{x}) = \begin{cases} \left(\mu(\mathrm{x}) - f(\mathrm{x}^+)\right)\Phi(Z) + \sigma(\mathrm{x})\,\phi(Z) & \text{si } \sigma(\mathrm{x}) > 0 \\ 0 & \text{si } \sigma(\mathrm{x}) = 0 \end{cases}$$

donde:

$$Z = \frac{\mu(\mathrm{x}) - f(\mathrm{x}^+)}{\sigma(\mathrm{x})}$$

- $\phi(\cdot)$: PDF de la normal estándar
- $\Phi(\cdot)$: CDF de la normal estándar
- $f(\mathrm{x}^+)$: mejor valor observado hasta ahora

Optimización Bayesiana — Algoritmo formal

Algoritmo 3: Optimización Bayesiana

Entrada: Función objetivo f, espacio Λ, presupuesto T, puntos iniciales n_0
Salida : Mejor configuración $\boldsymbol{\lambda}^*$

1 Evaluar f en n_0 puntos aleatorios $\rightarrow \mathcal{D}_0 = \{(\boldsymbol{\lambda}_i, y_i)\}_{i=1}^{n_0}$
2 **for** $t = 1$ **to** $T - n_0$ **do**
3 Ajustar un Proceso Gaussiano a $\mathcal{D}_{t-1}$
4 Calcular la función de adquisición $\alpha(\boldsymbol{\lambda})$ usando el GP
5 $\boldsymbol{\lambda}_t \leftarrow \arg\max_{\boldsymbol{\lambda} \in \Lambda} \alpha(\boldsymbol{\lambda})$
6 Evaluar $y_t = f(\boldsymbol{\lambda}_t)$ (entrenar modelo + CV)
7 $\mathcal{D}_t \leftarrow \mathcal{D}_{t-1} \cup \{(\boldsymbol{\lambda}_t, y_t)\}$
8 **end**
9 **return** $\boldsymbol{\lambda}^* = \arg\max_{(\boldsymbol{\lambda}, y) \in \mathcal{D}_T} y$

Nota: El paso 5 (optimizar α) es barato porque α tiene fórmula cerrada. El paso costoso es el 6 (evaluar f, que requiere entrenar el modelo).

Ejemplo paso a paso — Inicio

Simplificación: Fijamos $C = 1$ y optimizamos solo $\gamma \in [0{,}001, 10]$ (1D).

Ejemplo paso a paso — Inicio

Simplificación: Fijamos $C = 1$ y optimizamos solo $\gamma \in [0{,}001, 10]$ (1D).

Paso 0: Evaluaciones iniciales aleatorias ($n_0 = 2$):

Punto	γ	Accuracy
1	0.01	0.65
2	1.00	0.58

Ejemplo paso a paso — Inicio

Simplificación: Fijamos $C = 1$ y optimizamos solo $\gamma \in [0{,}001, 10]$ (1D).

Paso 0: Evaluaciones iniciales aleatorias ($n_0 = 2$):

Punto	γ	Accuracy
1	0.01	0.65
2	1.00	0.58

Ajustar GP inicial:

- El GP interpola los 2 puntos observados
- Alta incertidumbre en regiones no exploradas
- El GP "no sabe" qué pasa entre $\gamma = 0{,}01$ y $\gamma = 1$

Ejemplo paso a paso — Inicio

Simplificación: Fijamos $C = 1$ y optimizamos solo $\gamma \in [0{,}001, 10]$ (1D).

Paso 0: Evaluaciones iniciales aleatorias ($n_0 = 2$):

Punto	γ	Accuracy
1	0.01	0.65
2	1.00	0.58

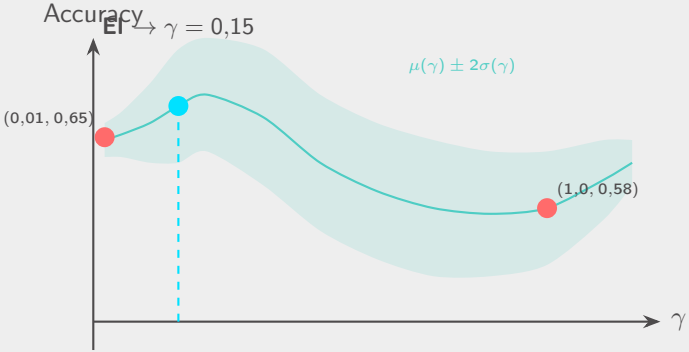
Ajustar GP inicial:

- El GP interpola los 2 puntos observados
- Alta incertidumbre en regiones no exploradas
- El GP "no sabe" qué pasa entre $\gamma = 0{,}01$ y $\gamma = 1$

$f(x^+) = 0{,}65$ (mejor observación hasta ahora, en $\gamma = 0{,}01$).

Ejemplo paso a paso — Iteración 1

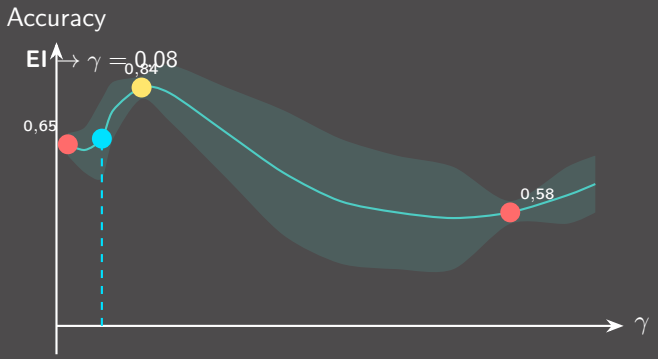
GP posterior con 2 observaciones. La función EI sugiere $\gamma = 0{,}15$.



Evaluación: $f(\gamma{=}0{,}15) = 0{,}84$. Nuevo mejor: $f(\mathrm{x}^{+}) = 0{,}84$.

Ejemplo paso a paso — Iteración 2

GP actualizado con 3 observaciones. EI sugiere $\gamma = 0{,}08$.



Evaluación: $f(\gamma{=}0{,}08) = 0{,}80$. El GP refina la región prometedora.
La incertidumbre **disminuye** cerca de las observaciones y el modelo **aprende** la forma.

Ejemplo paso a paso — Iteraciones 3–5

Resumen de **todas las evaluaciones** de la optimización bayesiana:

Iter.	Tipo	γ	Accuracy	Razón de EI
0	Inicial	0.010	0.65	—
0	Inicial	1.000	0.58	—
1	EI	0.150	0.84	Alta incertidumbre
2	EI	0.080	0.80	Explorar vecindad
3	EI	0.200	0.83	Explorar otro lado
4	EI	0.120	**0.87**	Refinar máximo
5	EI	0.110	0.86	Confirmar

Mejor resultado: $\gamma^* = 0{,}12$, accuracy $= 0{,}87$

Observaciones:

- El GP converge a la región óptima $\gamma \in [0{,}08, 0{,}20]$
- Solo 7 evaluaciones totales (vs. 9 en Grid/Random)
- Las últimas iteraciones **refinan** el óptimo, no exploran a ciegas

Conclusiones

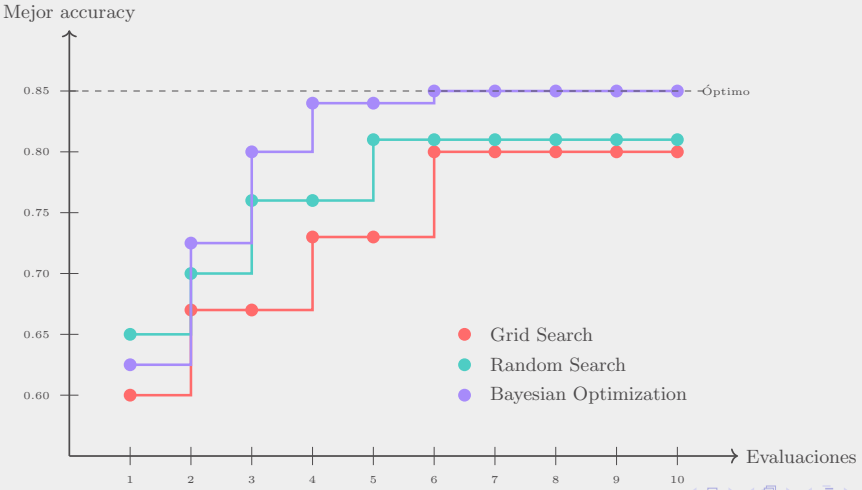
Resumen y comparación **de los tres métodos**

Comparación de los tres métodos

Aspecto	Grid Search	Random Search	Bayesian Opt.
Tipo de búsqueda	Exhaustiva	Aleatoria	Secuencial informada
Aprende de evaluaciones previas	No	No	Sí
Paralelizable	Sí (total)	Sí (total)	Limitado
Núm. evaluaciones	m^d (exponencial)	Fijo (presupuesto)	Fijo (presupuesto)
Garantía	Mejor en la grilla	Probabilística	Convergencia teórica
Costo por paso	Bajo	Bajo	Medio (ajustar GP)
Mejor para	Pocos hiperparám.	Exploración inicial	Evaluación costosa

En nuestro ejemplo: Grid Search $= 0{,}85$, Random Search $= 0{,}86$, Bayesian Opt. $= 0{,}87$.

Convergencia comparada



Convergencia comparada — Observación

Observación: La optimización bayesiana típicamente alcanza mejores resultados con **menos evaluaciones**, porque cada iteración usa la información acumulada para elegir el próximo punto de manera inteligente.

- **Grid Search:** costo fijo alto; la mejora depende del orden de evaluación en la grilla
- **Random Search:** mejora rápido al inicio pero se estanca al no aprender de evaluaciones previas
- **Bayesian Optimization:** converge de forma eficiente al dirigir la búsqueda hacia regiones prometedoras

¿Cuándo usar cada método?

Escenario	Recomendación
1–2 hiperparámetros, pocos valores	Grid Search: exhaustivo y simple
3–5 hiperparámetros, exploración rápida	Random Search: buena cobertura
Evaluación costosa (horas por modelo)	Bayesian Optimization: minimiza evaluaciones
Deep learning, muchos recursos	Random Search + refinamiento con BO
Necesidad de reproducibilidad	Grid Search: determinístico
Espacio categórico puro	Grid Search o Random Search
$>$ 10 **hiperparámetros**	Random Search (o métodos avanzados: TPE, BOHB)

Referencias

- Hastie, T., Tibshirani, R. & Friedman, J. (2009). *The Elements of Statistical Learning*, 2nd Ed., Cap. 7. Springer.
- Bergstra, J. & Bengio, Y. (2012). Random Search for Hyper-Parameter Optimization. *Journal of Machine Learning Research*, 13, 281–305.
- Snoek, J., Larochelle, H. & Adams, R.P. (2012). Practical Bayesian Optimization of Machine Learning Algorithms. *Advances in Neural Information Processing Systems* (NeurIPS).
- Rasmussen, C.E. & Williams, C.K.I. (2006). *Gaussian Processes for Machine Learning*. MIT Press.
- Shahriari, B. et al. (2016). Taking the Human Out of the Loop: A Review of Bayesian Optimization. *Proceedings of the IEEE*, 104(1), 148–175.

UTEC Posgrado

UTEC Posgrado